

EJEMPLO 7 – UNIDAD DE TRABAJO 5.

DISPARADORES O TRIGGERS (CONTINUACIÓN)

Empleando el constructor **BEGIN ... END**, se puede definir un disparador que ejecute sentencias múltiples. Dentro del bloque **BEGIN**, también pueden utilizarse otras sintaxis permitidas en rutinas almacenadas, tales como condicionales y bucles. Como sucede con los procedimientos o rutinas almacenadas, cuando se crea un disparador que ejecuta sentencias múltiples, **se hace necesario redefinir el delimitador de sentencias** si se ingresará el disparador a través del programa mysql, de forma que se pueda utilizar el carácter ';' dentro de la definición del disparador.

1.- El siguiente ejemplo ilustra estos aspectos. Siguiendo con la tabla **pruebas**, vamos a crear un disparador para **UPDATE**, que verifica los valores utilizados para actualizar cada columna, y modifica el valor para que se encuentre en un rango de 0 a 100. Esto debe hacerse en un disparador **BEFORE** porque los valores deben verificarse antes de emplearse para actualizar el registro.

```
\d //
CREATE TRIGGER upd_check BEFORE UPDATE ON pruebas
FOR EACH ROW
BEGIN
IF NEW.aumento < 0 THEN
    SET NEW.aumento = 0;
ELSEIF NEW.aumento > 100 THEN
    SET NEW.aumento = 100;
END IF;
END;
//
\d ;
```

1.1.- Vamos a probarlo:

```
UPDATE PRUEBAS SET aumento=200 WHERE numero=30;
SELECT * FROM pruebas where numero=30;
```

2.- Crear una nueva tabla llamada **pruebas2** que tenga los campos: cantidad, precio, total, todos de tipo int y crear un disparador para antes de insertar un registro, en el que se actualiza automáticamente el campo total a partir de la multiplicación del campo cantidad por el precio.

```
CREATE TABLE pruebas2( cantidad int, precio int, total int);
```

```
Create trigger ins_check before insert on pruebas2
for each row
set new.total = new.cantidad * new.precio
```

2.1.- Vamos a probarlo. Podemos insertar los registros de dos maneras:

```
INSERT INTO pruebas2 VALUES (100,25,null);
```

O

```
INSERT INTO pruebas2 (cantidad, precio) VALUES (200,9);
```

```
SELECT * FROM pruebas2;
```

3.- Vamos a ver otro ejemplo de disparador, en el que se guardará en una tabla auditoría las modificaciones que se hagan sobre los registros sobre una tabla llamada clientes:

3.1.- Creamos la tabla clientes e introducimos 5 registros:

```
CREATE TABLE clientes(  
  id INT NOT NULL auto_increment,  
  nombre VARCHAR(100),  
  seccion VARCHAR(20),  
  PRIMARY KEY(id));  
  
INSERT INTO clientes (nombre, seccion) VALUES  
('Miguel','informatica'),  
('Rosa','comida'),  
('Maria','ropa'),  
('Albert','informatica'),  
('Jordi','comida');  
  
SELECT * FROM clientes;
```

3.2.- Y creamos una tabla (auditoria_clientes) que será la que guardará los datos de la “auditoria”:

```
CREATE TABLE auditoria_clientes  
(  
  id int not null auto_increment,  
  nombre varchar(100),  
  anterior_seccion varchar(20),  
  nueva_seccion varchar(20),  
  usuario varchar(40),  
  modificado datetime,  
  primary key(id)  
);  
  
describe auditoria_clientes;
```

3.3.- Y finalmente creamos un trigger que se disparará cada vez que alguien modifique un dato de la tabla clientes y lo guardará en la tabla auditoría_clientes junto al nombre del usuario y la fecha en que se realizó la modificación. Compruébalo modificando la sección para algún cliente y viendo que se guardó en la tabla auditoría_clientes.

```
\d //
```

```
CREATE TRIGGER trg_auditoria_clientes
```

```
AFTER UPDATE ON clientes
FOR EACH ROW
BEGIN
```

```
INSERT INTO auditoria_clientes (nombre, anterior_seccion, nueva_seccion,
usuario, modificado)
VALUES (NEW.nombre, OLD.seccion, NEW.seccion, CURRENT_USER(), NOW());
```

```
END //
```

```
DELIMITER ;
```

```
UPDATE CLIENTES SET SECCION = 'Textil' WHERE nombre = 'Maria';
UPDATE CLIENTES SET SECCION = 'Frutería' WHERE nombre = 'Rosa';
SELECT * FROM AUDITORIA_CLIENTES;
```

4.- Crear una tabla denominada **pruebas3** que tenga los siguientes campos numéricos (numero1, numero2, total). Luego crear un disparador que calcule el total dependiendo del valor de la suma de los números 1 y 2. Si la suma es positiva, haremos que el total sea igual a la multiplicación de los números 1 y 2. Y si la suma es negativa que sea igual a la multiplicación de ambos números + 100.

```
CREATE TABLE pruebas3 (numero1 INT, numero2 INT, total INT);
```

Compruébalo con:

```
\d //
```

```
CREATE TRIGGER calcular_tiototal BEFORE INSERT ON pruebas3
FOR EACH ROW
BEGIN
```

```
    IF (NEW.numero1 + NEW.numero2) > 0 THEN
        SET NEW.total = NEW.numero1 * NEW.numero2;
```

```
    ELSE
```

```
        SET NEW.total = (NEW.numero1 * NEW.numero2) + 100;
```

```
    END IF;
```

```
END //
```

```
\d ;
```

```
INSERT INTO pruebas3 VALUES (18,10,NULL),(-100,90,NULL);
```

```
SELECT * FROM pruebas3;
```